



MACHINE LEARNING

CLASSIFICATION

BASIC CONCEPTS

Classification is a classic **data science task**, with roots in **machine learning**.

There are many different types of **machine learning techniques** that can be categorized based on:

- Whether or not they are **trained with human supervision**
 - **supervised**, **unsupervised**, **semi-supervised**, and **Reinforcement Learning**
- Whether or not they can **learn incrementally** on the fly
 - **batch** and **online** learning
- Whether they work by simply **comparing new data points to known data points**, or instead **detect patterns in the training data and build a predictive model**
 - **instance-based** and **model-based** learning

BASIC CONCEPTS

Classification is fall under the supervised learning type of machine learning.

- **Supervised learning**

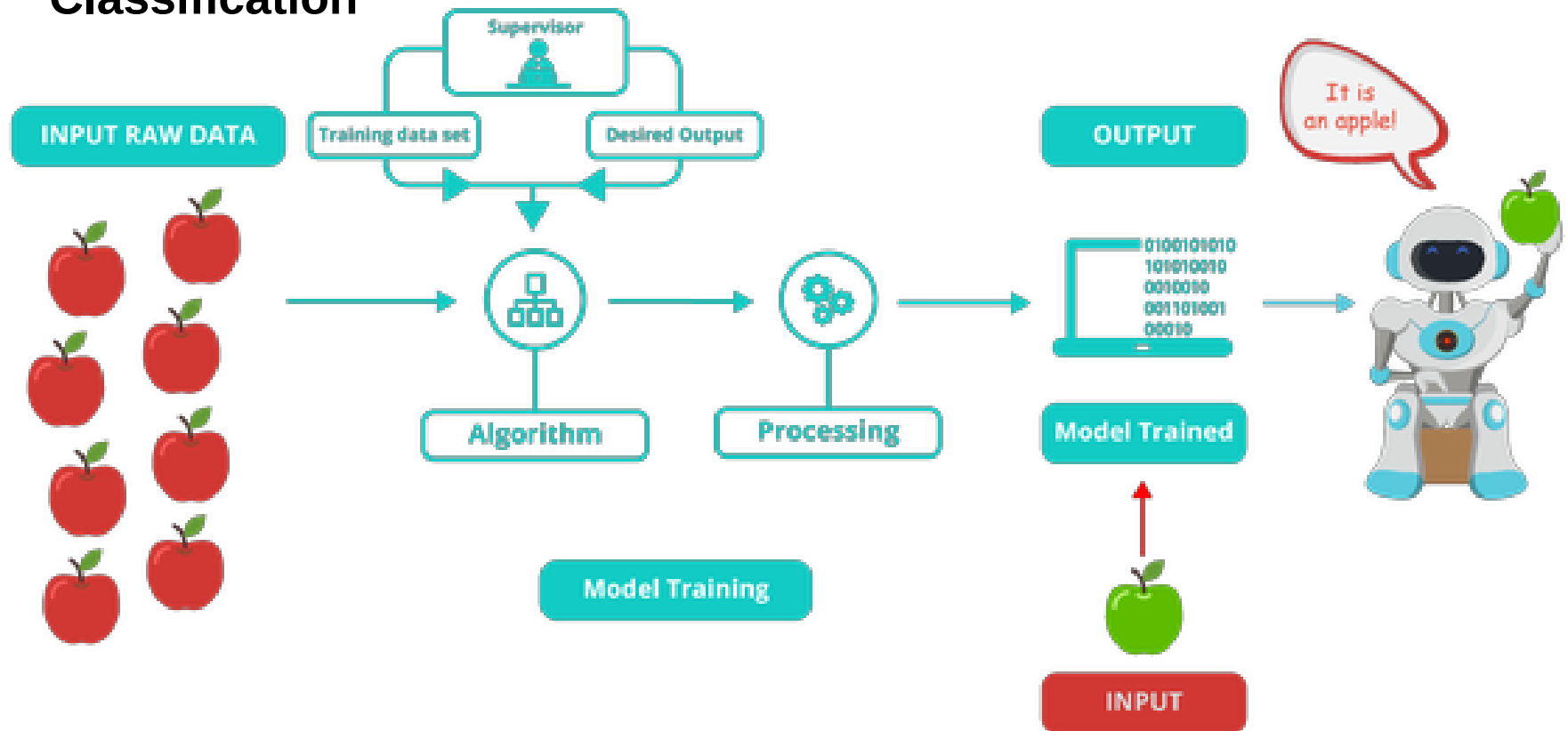
- **Supervision**: The training data (observations, measurements, ...) are **accompanied by labels** indicating the class of the observations
- New data is classified based on the training set.

- **Classification**

- **predicts** categorical **class labels** (*discrete* or *nominal*)
- classifies data based on the training set and the values, **class labels**, in a classifying attribute and uses it in classifying new data.
 - Need to constructs a classification model

BASIC CONCEPTS

Classification



BASIC CONCEPTS

Classification is “Techniques used to **predict group membership** for data instances”.

For example, given past records

- of weather, we wish to use classification to predict whether the weather on a particular day will be “**sunny**”, “**rainy**” or “**cloudy**”.
- of customers who switched to another supplier, we wish to predict which current customers are likely to do the same."

BASIC CONCEPTS

A machine learning classifier is a computational object that has two stages:

- It gets “trained.” It takes in its training data, which is a bunch of data points and the correct label associated with them, and **tries** to **learn some pattern for how the points map to the labels**.
- Once it has been trained, the classifier **acts** as **a function that takes in additional data points and outputs predicted classifications for them**. The prediction will be a specific label.
 - Some times, it will give a continuous-valued number that can be seen as a confidence score for a particular label.

BASIC CONCEPTS

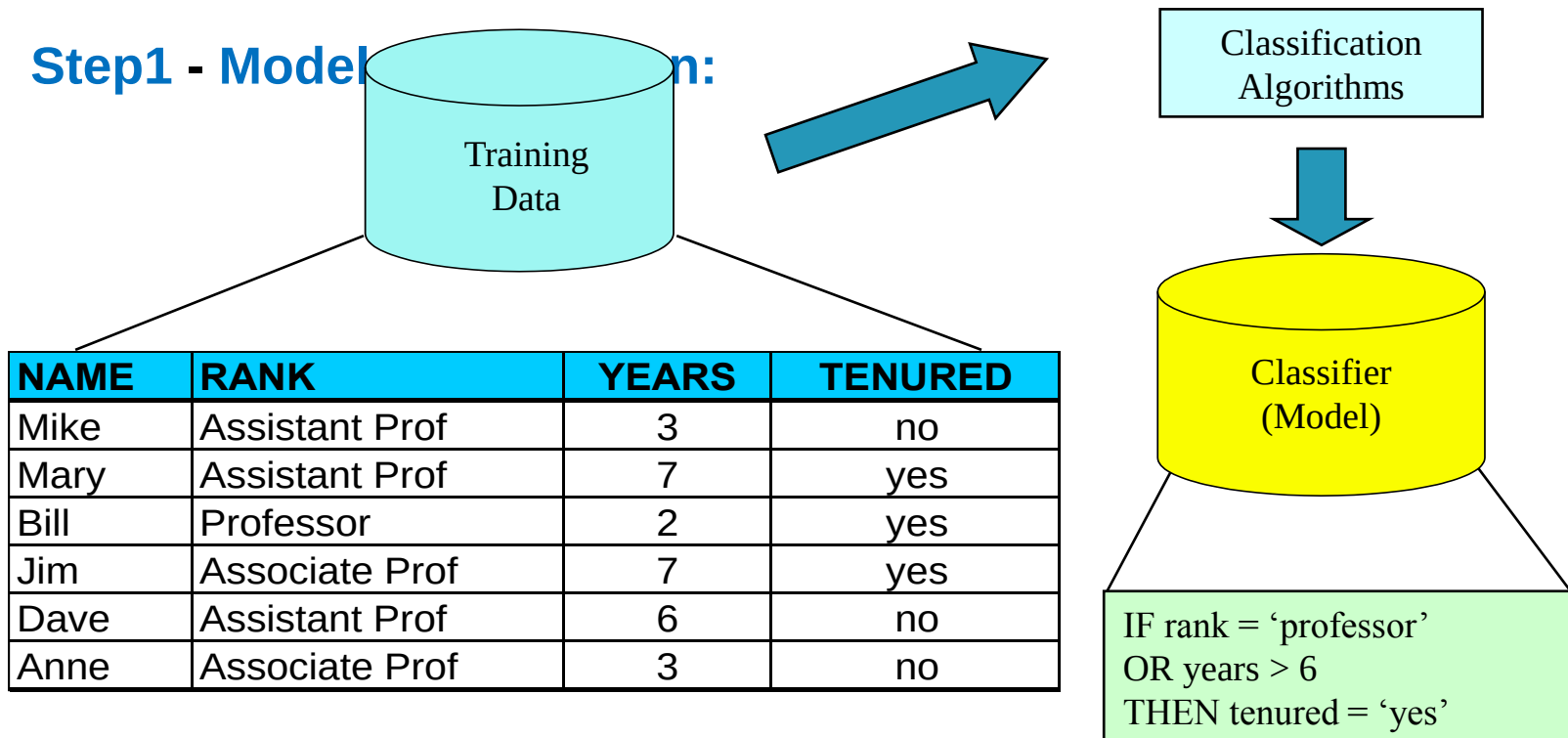
Classification is a **two-step process**:

Step1 - Model construction: describing a set of predetermined classes

- Each tuple, sample, is **assumed to belong to a predefined class**, as determined by the *class label* attribute
- The set of tuples used for model construction is *training set*
- The model is **represented as classification rules**
 - decision trees or mathematical formula

BASIC CONCEPTS

Classification is a two-step process:



BASIC CONCEPTS

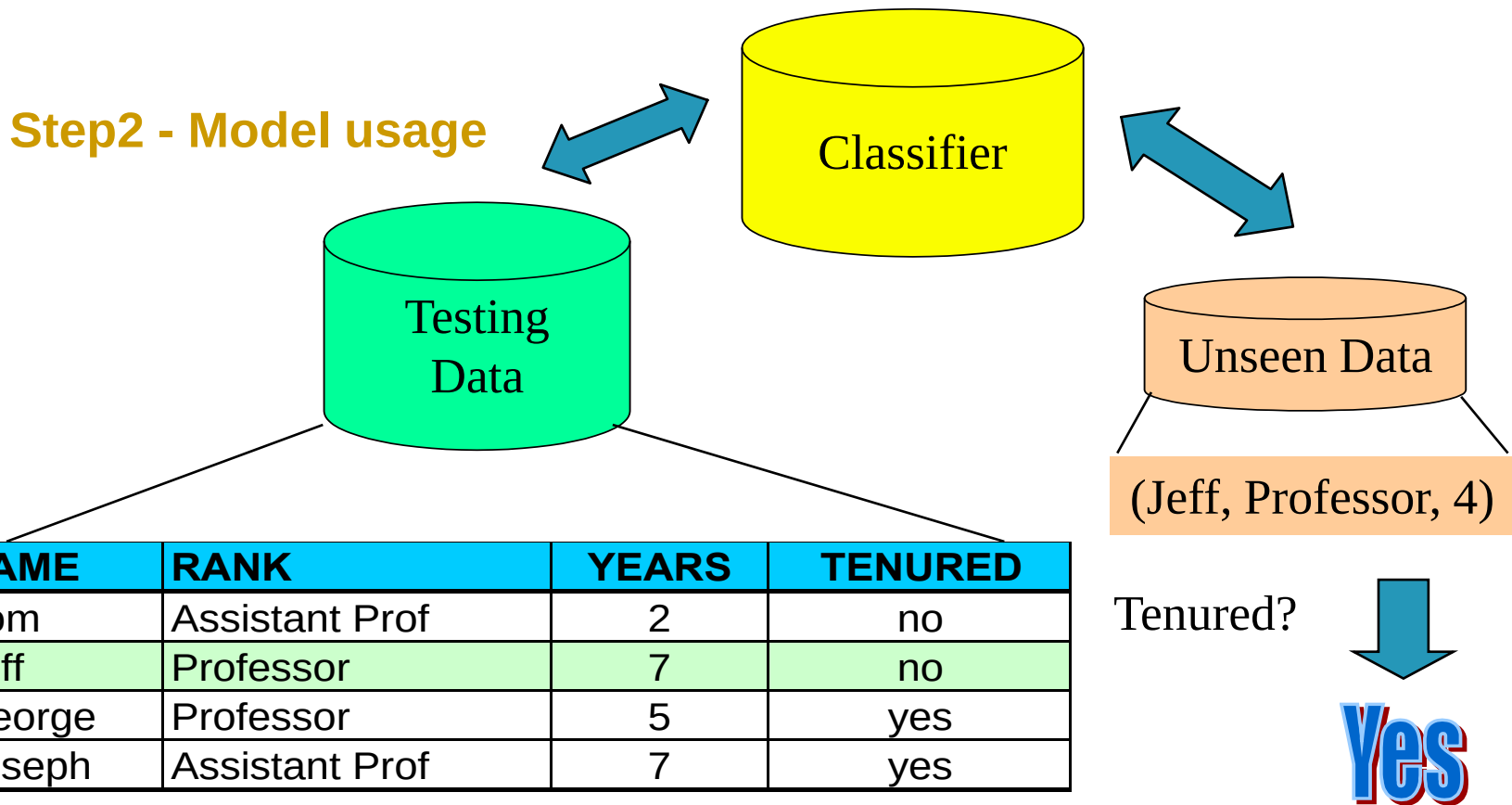
Classification is a two-step process:

Step2 - Model usage: for classifying future or unknown objects

- Estimate accuracy of the model
 - The known label of test sample is compared with the classified result from the model
 - Accuracy rate is the percentage of test set samples that are correctly classified by the model
 - Test set is independent of training set (otherwise over fitting)

BASIC CONCEPTS

Classification is a two-step process:



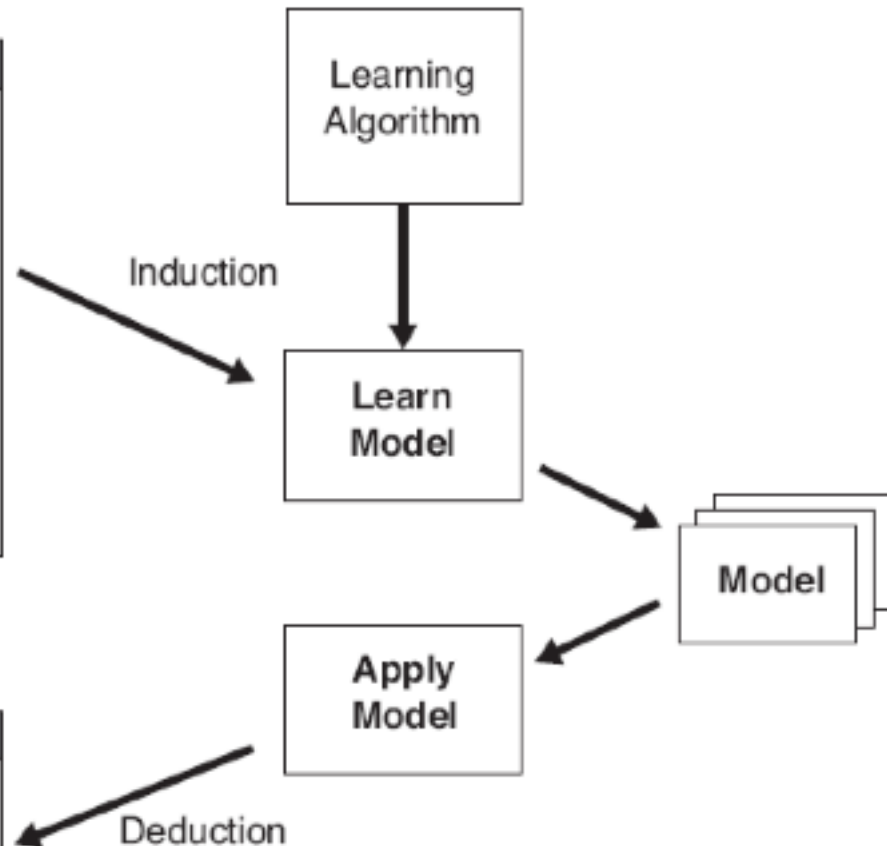
BASIC CONCEPTS

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Test Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?



General Approach for Building Classification Model

BASIC CONCEPTS

Accuracy:

- refers to the ability of a given classifier to correctly predict the class label of new or previously unseen data

Speed:

- refers to the computational costs involved in generating and using the given classifier.

Robustness:

- refers to the ability of the classifier to make correct predictions given noisy data or data with missing values.

BASIC CONCEPTS

Scalability:

- refers to the ability to construct the classifier efficiently given large amounts of data.

Interpretability:

- refers to the level of understanding and insight that is provided by the classifier .
- Interpretability is subjective and therefore more difficult to assess.

CLASSIFICATION ALGORITHMS

- Decision Tree Induction
- Rule-Based Classification
- k-Nearest Neighbors
- Naïve Bayesian Classifiers
- Backpropagation Neural Network
- Support Vector Machine
- ...etc

K-NEAREST NEIGHBORS KNN

K-NEAREST NEIGHBORS (KNN)

k-Nearest Neighbors (kNN) is known as **instance** based learning.

- It does not use any model to fit.
- It only based on memory.

kNN is a **classification algorithm** where the **result**, class, of new instance **is classified based on majority** of **k-Nearest Neighbors' category**.

kNN classifies a new instance based on attributes and training samples.

K-NEAREST NEIGHBORS

Given a query point, instance, it finds the closest k objects, training points, to the query point.

- K is a predetermined number

The classification is achieved by using majority vote among the class label of the k objects.

Any ties can be broken at random

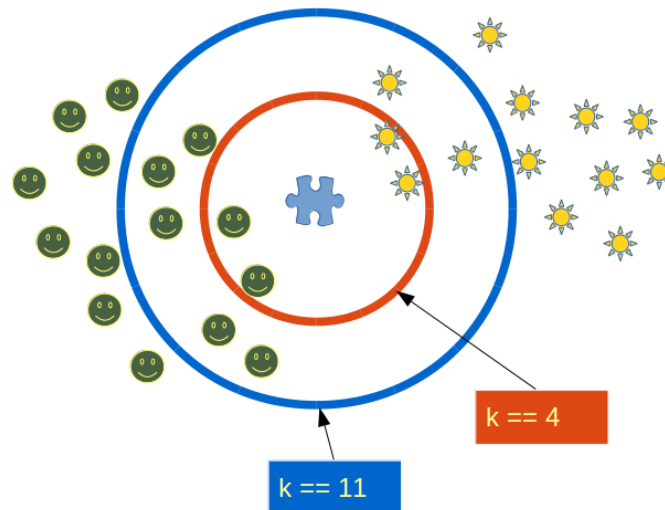
K-NEAREST NEIGHBORS

The main concept of kNN

- Given a new instance x ,
- find its nearest neighbor $\langle x', y' \rangle$
- Return y' as the class of x

To avoid any noise in decision use more than 1

🧩 == 😊 or 🧩 == ☀️ ?



K-NEAREST NEIGHBORS

All instances correspond to points are in the **n-D** space

The nearest neighbor is defined in terms of **similarity functions**

- Euclidean Distance or Manhattan distance.
- Assume that we have two data points, $X = (x_1, x_2, \dots, x_n)$ and $Y = (y_1, y_2, \dots, y_n)$

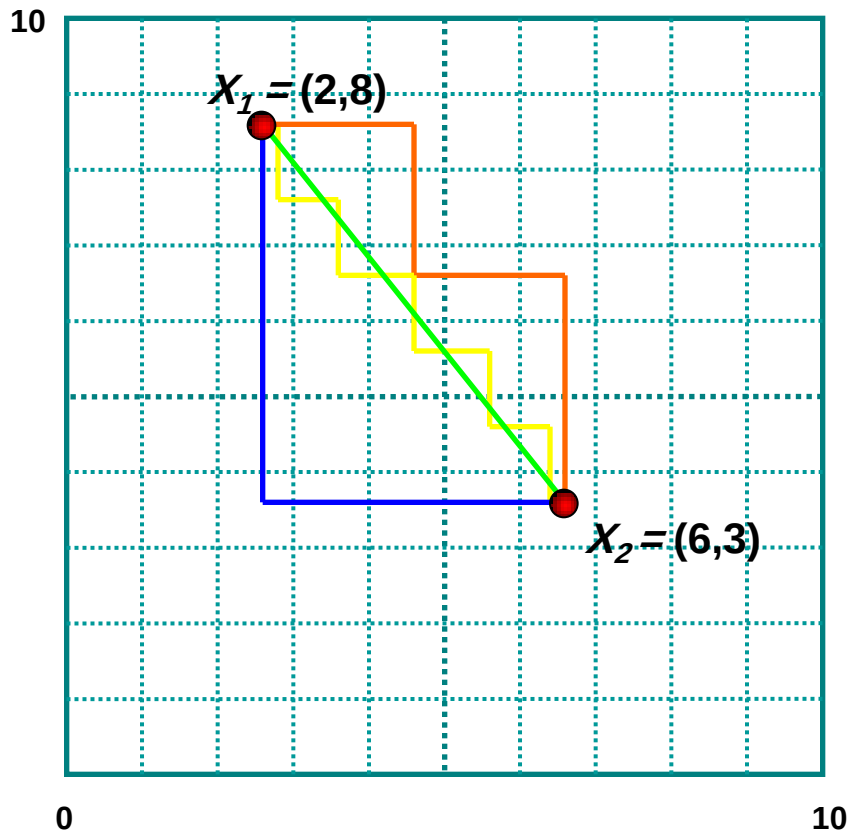
Euclidean Distance

$$d(X, Y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Manhattan Distance

$$d(X, Y) = \sum_{i=1}^n |x_i - y_i|$$

K-NEAREST NEIGHBORS



2D example

$$x_1 = (2, 8)$$

$$x_2 = (6, 3)$$

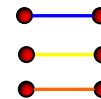
Euclidean distance

$$d(x_1, x_2) = \sqrt{(2 - 6)^2 + (8 - 3)^2} = \sqrt{41}$$



Manhattan distance

$$d(x_1, x_2) = |2 - 6| + |8 - 3| = 9$$



K-NEAREST NEIGHBORS ALGORITHM

Here is step by step on how to compute kNN algorithm:

1. **Determine** parameter **k**
 - the number of nearest neighbors.
2. **Calculate the distance between** the query-instance **and** all the training samples
 - Using Euclidean distance
3. **Sort the training set**, in ascending order, **based on the distance**
4. **Select the first k instances**
 - the K instances with minimum distances
5. **Use simple majority vote of the categories of nearest neighbors as the prediction value of the query-instance.**

K-NEAREST NEIGHBORS

EXAMPLE

Assume that we have data from the questionnaires survey with four training samples :

test a query-instance with $X_1 = 3$ and $X_2 = 7$

X_1	X_2	<i>Class</i>
7	7	Bad
7	4	Bad
3	4	Good
1	4	Good

K-NEAREST NEIGHBORS

EXAMPLE

1. Determine parameter K = number of nearest neighbors
 - for example use $K = 3$
2. Calculate the distance between the query-instance $(3, 7)$ and all the training samples
 - Use **Euclidean distance**

X_1	X_2	Distance	<i>Class</i>
7	7	$(7 - 3)^2 + (7 - 7)^2 = 16$	Bad
7	4	$(7 - 3)^2 + (4 - 7)^2 = 25$	Bad
3	4	$(3 - 3)^2 + (4 - 7)^2 = 9$	Good
1	4	$(1 - 3)^2 + (4 - 7)^2 = 13$	Good

K-NEAREST NEIGHBORS

EXAMPLE

- Sort the training set, in ascending order, based on the distance

X_1	X_2	Distance	<i>Class</i>
3	4	$(3 - 3)^2 + (4 - 7)^2 = 9$	Good
1	4	$(1 - 3)^2 + (4 - 7)^2 = 13$	Good
7	7	$(7 - 3)^2 + (7 - 7)^2 = 16$	Bad
7	4	$(7 - 3)^2 + (4 - 7)^2 = 25$	Bad

K-NEAREST NEIGHBORS

EXAMPLE

4. Select the first K instances, K=3

X_1	X_2	Distance	<i>Class</i>
3	4	$(3 - 3)^2 + (4 - 7)^2 = 9$	Good
1	4	$(1 - 3)^2 + (4 - 7)^2 = 13$	Good
7	7	$(7 - 3)^2 + (7 - 7)^2 = 16$	Bad
7	4	$(7 - 3)^2 + (4 - 7)^2 = 25$	Bad

K-NEAREST NEIGHBORS

EXAMPLE

5. Use simple majority vote of the category of nearest neighbors as the prediction value of the query instance.

- We have 2 Good and 1 Bad, then the new query-instance $(3, 7)$ belongs to Good category.

X_1	X_2	Distance	<i>Class</i>
3	4	$(3 - 3)^2 + (4 - 7)^2 = 9$	Good
1	4	$(1 - 3)^2 + (4 - 7)^2 = 13$	Good
7	7	$(7 - 3)^2 + (7 - 7)^2 = 16$	Bad
7	4	$(7 - 3)^2 + (4 - 7)^2 = 25$	Bad

K-NEAREST NEIGHBORS

CATEGORICAL VARIABLE

If we have a categorial attributes.

Use 0, 1 distance:

- for each attribute, **add 1** if the instances **differ** in that attribute and **otherwise add 0**

$$d_{0/1} = \sum_{i=1}^d \delta(x_i^a \neq x_i^b)$$

K-NEAREST NEIGHBORS

SCALING ISSUE

Attributes may have to be scaled to prevent distance measures from being dominated by one of the attributes

- **Solution:** Normalize the attributes to put it in an equal/equivalent scales.
 - for example: use min-max normalization to make all values between 0 and 1

userId	Calls Duration (Minutes)	SMS (Total)	Data Counter (MB)
1	25000	24	4
2	40000	27	5
3	55000	32	7
4	27000	25	5
5	53000	30	5

userId	Call Duration (Minutes)	SMS (Total)	Data Counter (MB)
1	0.0000333	0.1000000	0.2000000
2	0.5000000	0.4000000	0.4000000
3	1.0000000	0.9000000	0.8000000
4	0.0700000	0.2000000	0.4000000
5	0.9300000	0.7000000	0.4000000

K-NEAREST NEIGHBORS

STRENGTH AND WEAKNESS

Advantage

- Robust to noisy training data
- Effective if the training data is large

Disadvantage

- Need to determine K, subjective issue.
- Distance based learning is not clear
 - which type of distance to use, Euclidean distance or Manhattan distance, and
 - which attribute to use to produce the best results. Shall we use all attributes or certain attributes only?
- Computation cost is quite high because we need to compute distance of each query instance to all training samples.

KNN – PYTHON'S LIBRARIES

```
# Load/read the dataset from CSV file
iris_data = pd.read_csv('iris.csv')
# print(iris_data.head())

# extract features from dataset
features = iris_data.drop(['variety'], axis=1)
# where variety is name of the target attribute
# print(features.head())

# extract labels from dataset
labels = iris_data.variety
# print(labels.head())

# using k-Nearest Neighbors
from sklearn.neighbors import NearestNeighbors as knn
model = knn(5) # or can use model = NearestNeighbors(5)
model.fit(features, labels)
test = np.array([5.0, 3.6, 1.2, 0.17]).reshape(1,-1)
predicts=model.kneighbors(test,5)
print(predicts)
```

NAÏVE BAYES CLASSIFICA TION

NAÏVE BAYES

Naive Bayes models are a group of **extremely fast** and **simple classification** algorithms that are often **suitable for very high-dimensional datasets**.

Because they **are so fast** and **have so few tunable parameters**, they end up being **very useful** as a **quick-and-dirty baseline** for a classification problem.

Naive Bayes classifiers are **built on Bayesian classification methods**. These **rely on Bayes's theorem**, which is an equation **describing the relationship of conditional probabilities of statistical quantities**.

NAÏVE BAYES

This is where the “**naive**” in “**naive Bayes**” comes in: if we make very **naive assumptions** about the generative model for each label, we can find a rough approximation of the generative model for each class, and then proceed with the Bayesian classification.

- Different types of naive Bayes classifiers rest on different naive assumptions about the data,

The naive Bayes classification algorithm was built on the assumption of **independent events**, to avoid **the need to compute these messy conditional probabilities**.

- If everything was independent, the world of probability would be a much simpler place.

NAÏVE BAYES FORMULATION

In Bayesian classification, we're interested in **finding the probability of a label given some observed features**, which we can write as $P(\text{Class} \mid \text{features})$.

Bayes's theorem tells us how to express this in terms of quantities we can compute more directly.

- Suppose we wish to classify the vector $X = (x_1, \dots, x_n)$ into one of m classes $C_1, \dots, C_m$.

$$p(C_i \mid X) = \frac{p(C_i) \cdot p(X \mid C_i)}{p(X)}$$

$$C(X) = \arg \max_{i=1, \dots, m} \frac{p(C_i) \cdot p(X \mid C_i)}{p(X)} = \arg \max_{i=1, \dots, m} p(C_i) \cdot p(X \mid C_i).$$

NAÏVE BAYES FORMULATION

$$p(C_i|X) = \frac{p(C_i) \cdot p(X|C_i)}{p(X)}$$

$$C(X) = \arg \max_{i=1,\dots,m} \frac{p(C_i) \cdot p(X|C_i)}{p(X)} = \arg \max_{i=1,\dots,m} p(C_i) \cdot p(X|C_i).$$

Where

- $p(C_i|X)$ is Posterior Probability
- $p(X|C_i)$ is Likelihood
- $p(C)$ is Class Prior Probability
- $p(X)$ is a Predictor Probability

NAÏVE BAYES EXAMPLE 1

Assume that we have the following dataset, where **Beach** is the **target class**.

Day	Outlook	Temp	Humidity	Beach?
1	Sunny	High	High	Yes
2	Sunny	High	Normal	Yes
3	Sunny	Low	Normal	No
4	Sunny	Mild	High	Yes
5	Rainy	Mild	Normal	No
6	Rainy	High	High	No
7	Rainy	Low	Normal	No
8	Cloudy	High	High	No
9	Cloudy	High	Normal	Yes
10	Cloudy	Mild	Normal	No

$p(X \text{Beach?})$		
Outlook	Yes	No
Sunny	3/4	1/6
Rainy	0/4	3/6
Cloudy	1/4	2/6
Temperature	Yes	No
Low	0/4	2/6
Mild	1/4	2/6
High	3/4	2/6
Humidity	Yes	No
Normal	2/4	4/6
High	2/4	2/6
$p(\text{Beach?})$	4/10	6/10

NAÏVE BAYES

EXAMPLE 1

What is the class of the query-instance (Sunny, Mild, High)?

$$C(X) = \arg \max_{i=1, \dots, m} \frac{p(C_i) \cdot p(X|C_i)}{p(X)} = \arg \max_{i=1, \dots, m} p(C_i) \cdot p(X|C_i).$$

$$p(\text{Yes} | (\text{Sunny}, \text{Mild}, \text{High})) = p(\text{Yes}) * P(\text{Sunny} | \text{Yes}) * P(\text{Mild} | \text{Yes}) * P(\text{High} | \text{Yes})$$

$$p(\text{Yes} | (\text{Sunny}, \text{Mild}, \text{High})) = (4/10) * (3/4) * (1/4) * (2/4) = \mathbf{0.0375}$$

$$p(\text{No} | (\text{Sunny}, \text{Mild}, \text{High})) = p(\text{No}) * P(\text{Sunny} | \text{No}) * P(\text{Mild} | \text{No}) * P(\text{High} | \text{No})$$

$$p(\text{No} | (\text{Sunny}, \text{Mild}, \text{High})) = (6/10) * (1/6) * (2/6) * (2/6) = \mathbf{0.0111}$$

Since $0.0375 > 0.0111$, naive Bayes is telling us to hit the beach.

- I.e. The class of query instance (Sunny, Mild, High) is Yes

NAÏVE BAYES

EXAMPLE 2

Use the following dataset to find the the class of (1, 2, 2).

Sample	A1	A2	A3	Class
1	1	2	1	1
2	0	0	1	1
3	2	1	2	2
4	1	2	1	2
5	0	1	2	1
6	2	2	2	2
7	1	0	1	1
8	2	1	1	3
9	1	1	2	3
10	2	2	1	3

$p(X Class)$			
A1	1	2	3
0	2/4	0/3	0/3
1	2/4	1/3	1/3
2	0/4	2/3	2/3
A2	1	2	3
0	2/4	0/3	0/3
1	1/4	1/3	2/3
2	1/4	2/3	1/3
A3	1	2	3
1	3/4	1/3	2/3
2	1/4	2/3	1/3
$p(Class)$	4/10	3/10	3/10

NAÏVE BAYES

EXAMPLE 2

$$p(1|(1, 2, 2)) = p(1) * p(1|1) * p(2|1) * p(2|1)$$

$$p(1|(1, 2, 2)) = \left(\frac{4}{10}\right) * \left(\frac{2}{4}\right) * \left(\frac{1}{4}\right) * \left(\frac{1}{4}\right)$$

$$p(1|(1, 2, 2)) = \mathbf{0.025}$$

$$p(2|(1, 2, 2)) = p(2) * p(1|2) * p(2|2) * p(2|2)$$

$$p(2|(1, 2, 2)) = \left(\frac{3}{10}\right) * \left(\frac{1}{3}\right) * \left(\frac{2}{3}\right) * \left(\frac{2}{3}\right)$$

$$p(2|(1, 2, 2)) = \mathbf{0.0444}$$

$$p(3|(1, 2, 2)) = p(3) * p(1|3) * p(2|3) * p(2|3)$$

$$p(3|(1, 2, 2)) = \left(\frac{3}{10}\right) * \left(\frac{1}{3}\right) * \left(\frac{1}{3}\right) * \left(\frac{1}{3}\right)$$

$$p(3|(1, 2, 2)) = \mathbf{0.0111}$$

$p(X Class)$			
A1	1	2	3
0	2/4	0/3	0/3
1	2/4	1/3	1/3
2	0/4	2/3	2/3
A2	1	2	3
0	2/4	0/3	0/3
1	1/4	1/3	2/3
2	1/4	2/3	1/3
A3	1	2	3
1	3/4	1/3	2/3
2	1/4	2/3	1/3
$p(Class)$	4/10	3/10	3/10

Then (1, 2, 2) belongs to Class 2

WHEN TO USE NAIVE BAYES

Naive Bayesian classifiers make such stringent assumptions about data, so they have several advantages:

- They are extremely fast for both training and prediction
- They provide straightforward probabilistic prediction
- They are often very easily interpretable
- They have very few (if any) tunable parameters

These advantages of Naive Bayesian classifier is often a good choice as an initial baseline classification.

WHEN TO USE NAIVE BAYES

Because Naive Bayesian classifiers make such stringent assumptions about data, they will generally **not perform as well as** a more **complicated model**.

But it **tends to perform well** in one of the following situations:

- When the **naive assumptions actually match the data**
 - very rare in practice
- For **very well-separated categories**, when model complexity is less important
- For **very high-dimensional data**, when model complexity is less important
 - The last two points seem distinct, but they actually are related: as the dimension of a dataset grows, it is much less likely for any two points to be found close together (after all, they must be close in *every single dimension* to be close overall).

NAÏVE BAYES – PYTHON'S LIBRARIES

```
# Load/read the dataset from CSV file
iris_data = pd.read_csv('iris.csv')
# print(iris_data.head())

# extract featuers from dataset
featuers = iris_data.drop(['variety'], axis=1)
# where variety is name of the target attribute
# print(featuers.head())

# extract labels from dataset
labels = iris_data.variety
# print(labels.head())

# Naive Bayes
from sklearn.naive_bayes import GaussianNB as gnb
model = gnb()
model.fit(featuers, labels)
test = np.array([5.0, 3.6, 1.2, 0.17]).reshape(1,-1)
predicts=model.predict(test)
print(predicts)
```

DECISION TREE INDUCTION

DECISION TREE INDUCTION

Decision Tree Induction is the learning of decision trees from training set.

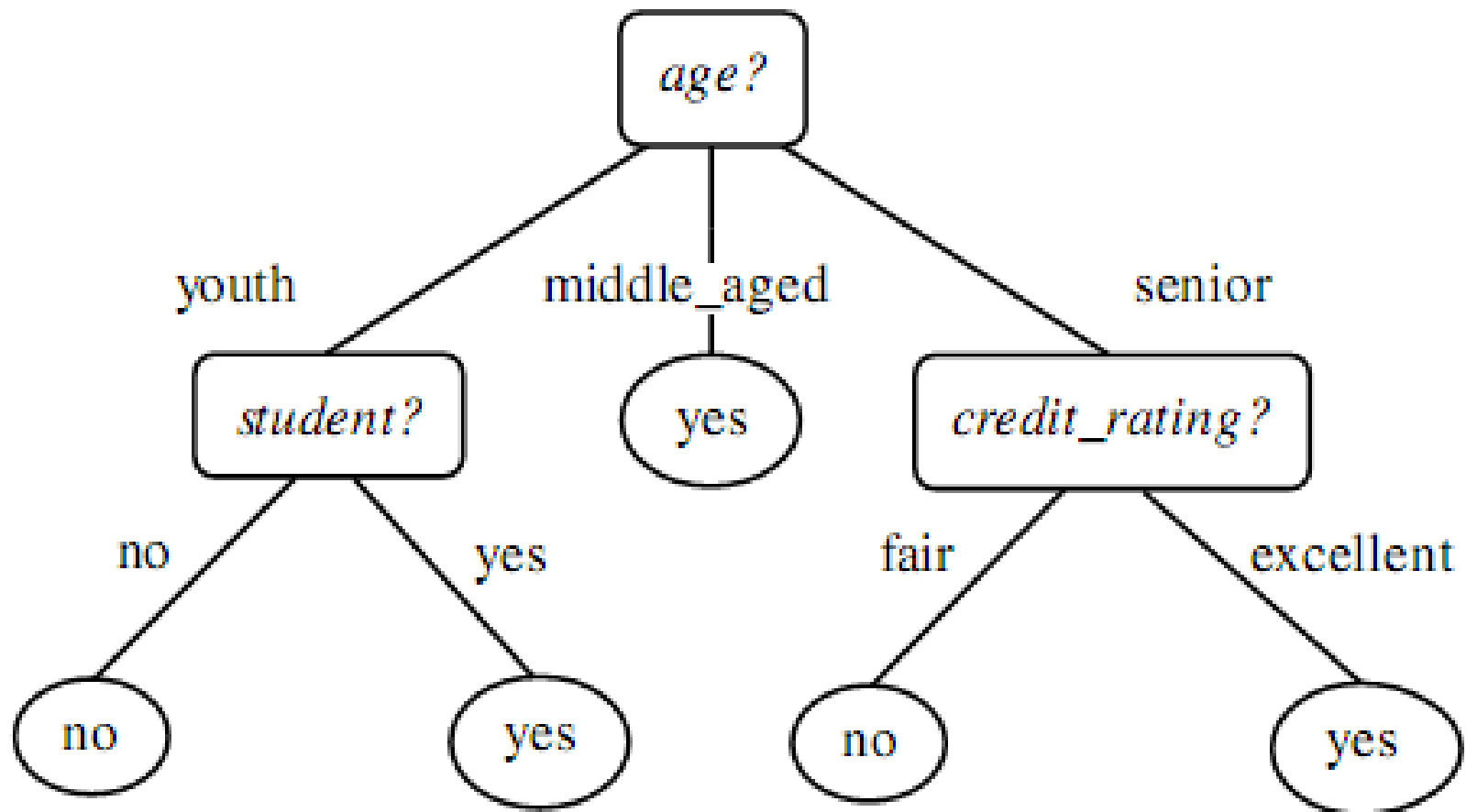
A decision tree is a flowchart-like tree structure, where

- each *internal node* (non leaf node) denotes a test on an attribute,
- each *branch* represents an outcome of the test, and
- each *leaf node* (or *terminal node*) holds a class label.

DECISION TREE INDUCTION

RID	age	income	student	credit rating	Buy Computer?
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle aged	medium	no	excellent	yes
13	middle aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

DECISION TREE INDUCTION



DECISION TREE INDUCTION ALGORITHM (C4.5)

Basic algorithm (C4.5): the tree is constructed in a top-down recursive divide-and-conquer manner

- greedy algorithm
- the successor of ID3.
- At start, all the training examples are at the root
- Attributes are categorical
 - if continuous-valued, they are discretized in advance
- Dataset's instances are partitioned recursively based on selected attributes
- Test attributes are selected on the basis of a heuristic or statistical measure
 - e.g., information gain

DECISION TREE INDUCTION ALGORITHM (C4.5)

Conditions for stopping partitioning

- All samples for a given node belong to the same class
- There are no remaining attributes for further partitioning
 - majority voting is employed for classifying the leaf
- There are no samples left

ATTRIBUTE SELECTION

INFORMATION GAIN

Select the **attribute with the highest information gain**

- Let p_i be the **probability** that an arbitrary **tuple in D** belongs to **class C_i** , estimated by $|C_i, D|/|D|$
- **Expected information (Entropy)** needed to classify a **tuple in D**:

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$

- **Information needed** (after using attribute A to split D into v partitions) to classify D :

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

- Information gained by branching on attribute A

$$Gain(A) = Info(D) - Info_A(D)$$

ATTRIBUTE SELECTION INFORMATION GAIN

Classes:

Class P: yes, and **Class N:** no

#of yes = 9, #of no = 5

$$Info(D) = I(9,5) = -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right)$$

$$Info(D) = 0.940$$

age	Yes	No	$I(Yes_i, No_i)$
youth	2	3	0.971
middle aged	4	0	0
senior	3	2	0.971

$$Info_{age}(D) = \frac{5}{14} I(2,3) + \frac{4}{14} I(4,0) + \frac{5}{14} I(3,2)$$

$$Info_{age} = 0.694$$

RI D	age	income	student	credit rating	Buys Computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle aged	medium	no	excellent	yes
13	middle aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

ATTRIBUTE SELECTION

INFORMATION GAIN

income	P_i	N_i	$I(P_i, N_i)$
high	2	2	0.811
medium	4	2	0.918
low	3	1	1

$$Info_{income}(D) = \frac{4}{14} I(2,2) + \frac{6}{14} I(4,2) + \frac{4}{14} I(3,1)$$

$$= 0.916$$

student	P_i	N_i	$I(P_i, N_i)$
yes	5	2	0.863
no	3	4	0.985

$$Info_{student}(D) = \frac{7}{14} I(5,2) + \frac{7}{14} I(3,4)$$

$$= 0.789$$

credit rating	P_i	N_i	$I(P_i, N_i)$
fair	6	2	0.811
excellent	3	3	1

$$Info_{credit_rating}(D) = \frac{8}{14} I(6,2) + \frac{6}{14} I(3,3)$$

$$= 0.892$$

ATTRIBUTE SELECTION INFORMATION GAIN

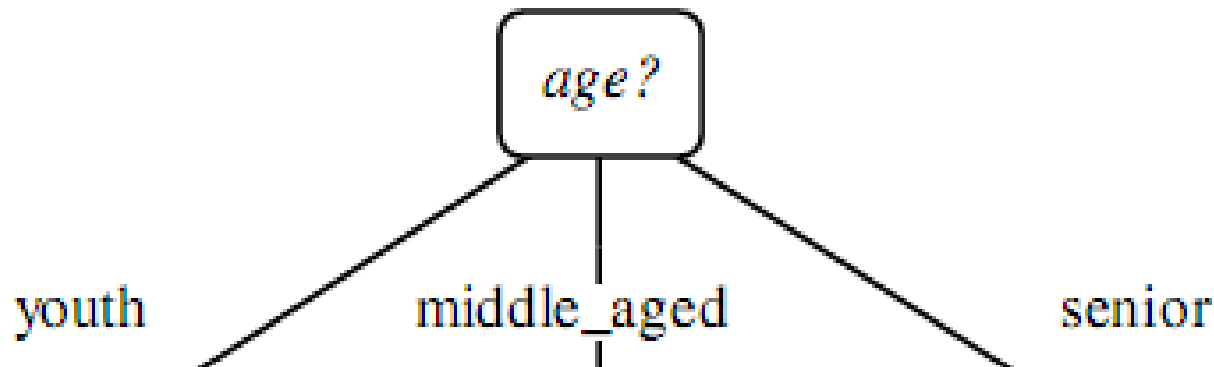
$$\text{Gain}(\text{age}) = \text{Info}(D) - \text{Info}_{\text{age}}(D) = 0.246$$

and similarly

$$\text{Gain}(\text{income}) = 0.029$$

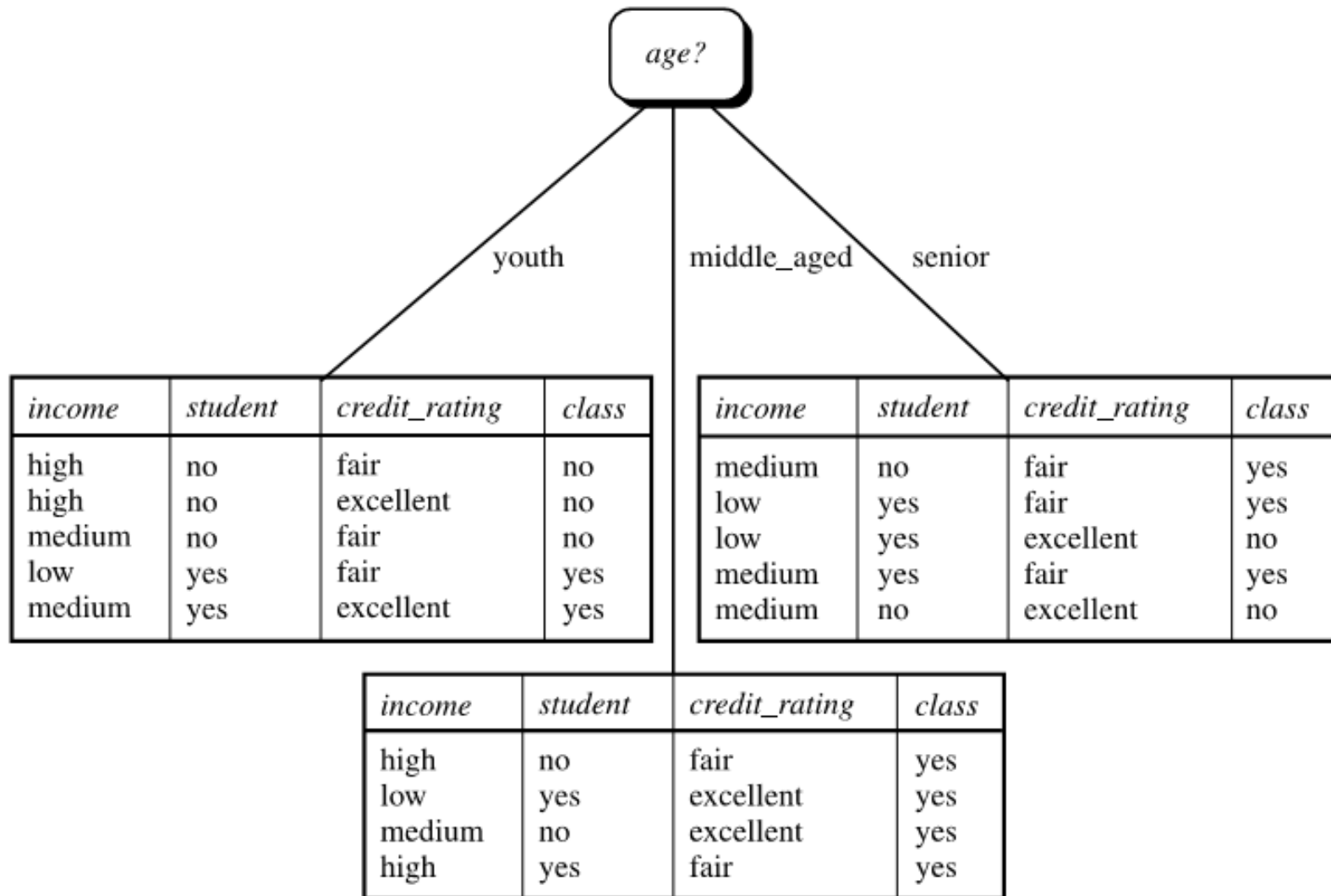
$$\text{Gain}(\text{student}) = 0.151$$

$$\text{Gain}(\text{credit_rating}) = 0.048$$



ATTRIBUTE SELECTION

INFORMATION GAIN



ATTRIBUTE SELECTION INFORMATION GAIN

Now, the dataset must be divided according to age then repeat the previous work as follow:

For *age = youth*, $I(2,3) = 0.971$

income	P_i	N_i	$I(P_i, N_i)$
high	0	2	0
medium	1	1	1
low	1	0	0

student	P_i	N_i	$I(P_i, N_i)$
yes	2	0	0
no	0	3	0

cre rating	P_i	N_i	$I(P_i, N_i)$
fair	1	2	0.918
excellent	1	1	1

$\text{Info}_{\text{income}} = 0.4$, $\text{Info}_{\text{student}} = 0$, $\text{Info}_{\text{creditrating}} = 0.951$

$\text{Gain}_{\text{income}} = 0.571$, $\text{Gain}_{\text{student}} = 0.971$, $\text{Gain}_{\text{creditrating}} = 0.02$

ATTRIBUTE SELECTION INFORMATION GAIN

What is the best split-point for continuous values attributes?

- First sort the values of A in increasing order.
 - Typically, the midpoint between each pair of adjacent values is considered as a possible split-point.
 - the midpoint between the values a_i and a_{i+1} of A is $\frac{a_i + a_{i+1}}{2}$
- If the values of A are sorted in advance, then determining the best split for A requires only one pass through the values.

ATTRIBUTE SELECTION

GAIN RATIO

The **information gain** measure is *biased* toward tests with many outcomes.

Gain ratio has been used to overcome the problem (normalization to information gain)

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left(\frac{|D_j|}{|D|} \right)$$

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo(A)}$$

The attribute **with the maximum gain ratio** is **selected** as the splitting attribute.

ATTRIBUTE SELECTION

OTHER ATTRIBUTE SELECTION MEASURES

Gini Index: biased to multivalued attributes and has difficulty when number of classes is large

CHAID: a popular decision tree algorithm, measure based on χ^2 test for independence

CART: finds multivariate splits based on a linear combination of attributes.

Which is the best measure for attribute selection?

- Most give good results, none is significantly superior than others

DECISION TREE INDUCTION OVERFITTING PROBLEM

Overfitting induced that a tree may over-fit the training data

- Too many branches,
 - some may **reflect anomalies** due to noise or outliers
- Poor accuracy for unseen samples

Two approaches to avoid overfitting

- **Pre-pruning**: Halt tree construction early—do not split a node if this would result in the goodness measure falling below a threshold
 - Difficult to choose an appropriate threshold
- **Post-pruning**: Remove branches from a “fully grown” tree—get a sequence of progressively pruned trees
 - Use a set of data different from the training data to decide which is the “best pruned tree”

DECISION TREE – PYTHON'S LIBRARIES

```
# Load/read the dataset from CSV file
iris_data = pd.read_csv('iris.csv')
# print(iris_data.head())

# extract features from dataset
features = iris_data.drop(['variety'], axis=1)
# where variety is name of the target attribute
# print(features.head())

# extract labels from dataset
labels = iris_data.variety
# print(labels.head())

# Decision Tree Classifier
from sklearn.tree import DecisionTreeClassifier as dt
model = dt(random_state=1)
model.fit(features, labels)
test = np.array([5.0, 3.6, 1.2, 0.17]).reshape(1,-1)
predicts=model.predict(test)
print(predicts)
```

NEURAL NETWORKS

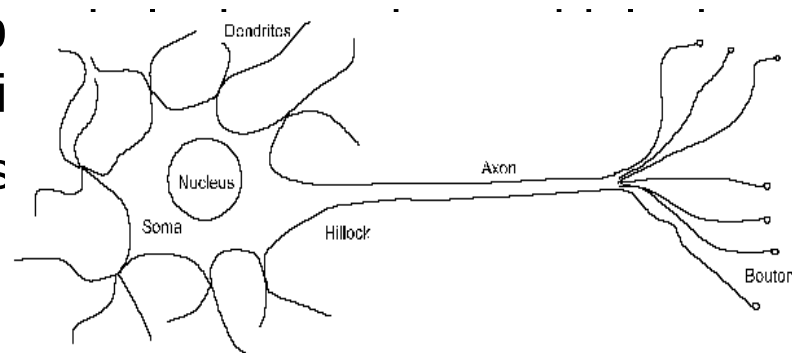
NEURAL NETWORKS

BASIC CONCEPT

Neural Network is a set of connected input/output units where each connection has a weight associated with it

- During the **learning phase**, the network learns by **adjusting the weights** so as to be able to predict the correct class label of the input tuples
- Also referred to as connectionist learning due to the connections between units

- It Started by p
test computati
- It is a s
body.

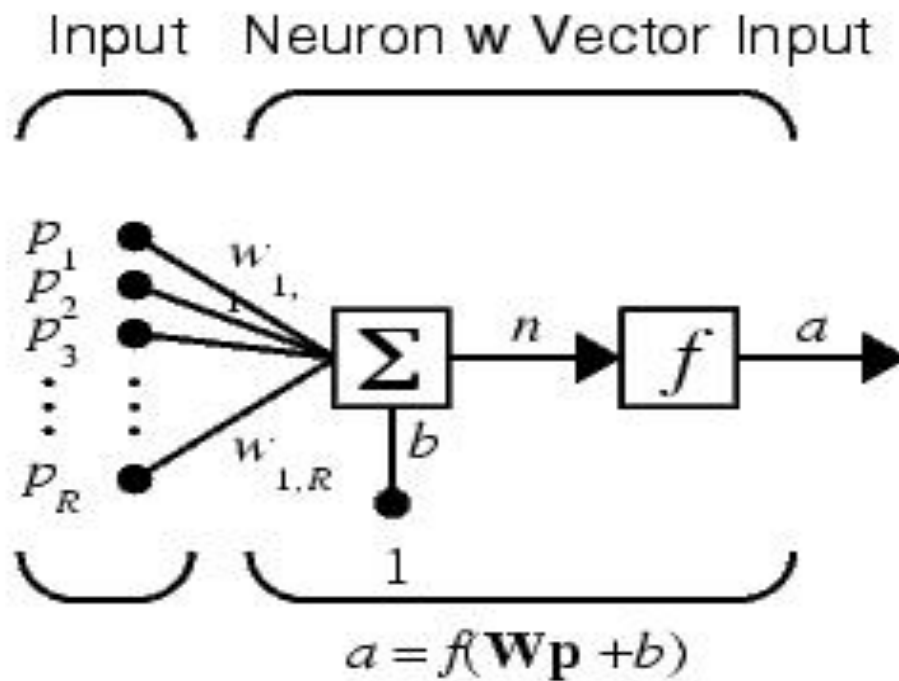


o develop and
in the human

NEURAL NETWORKS

BASIC CONCEPT

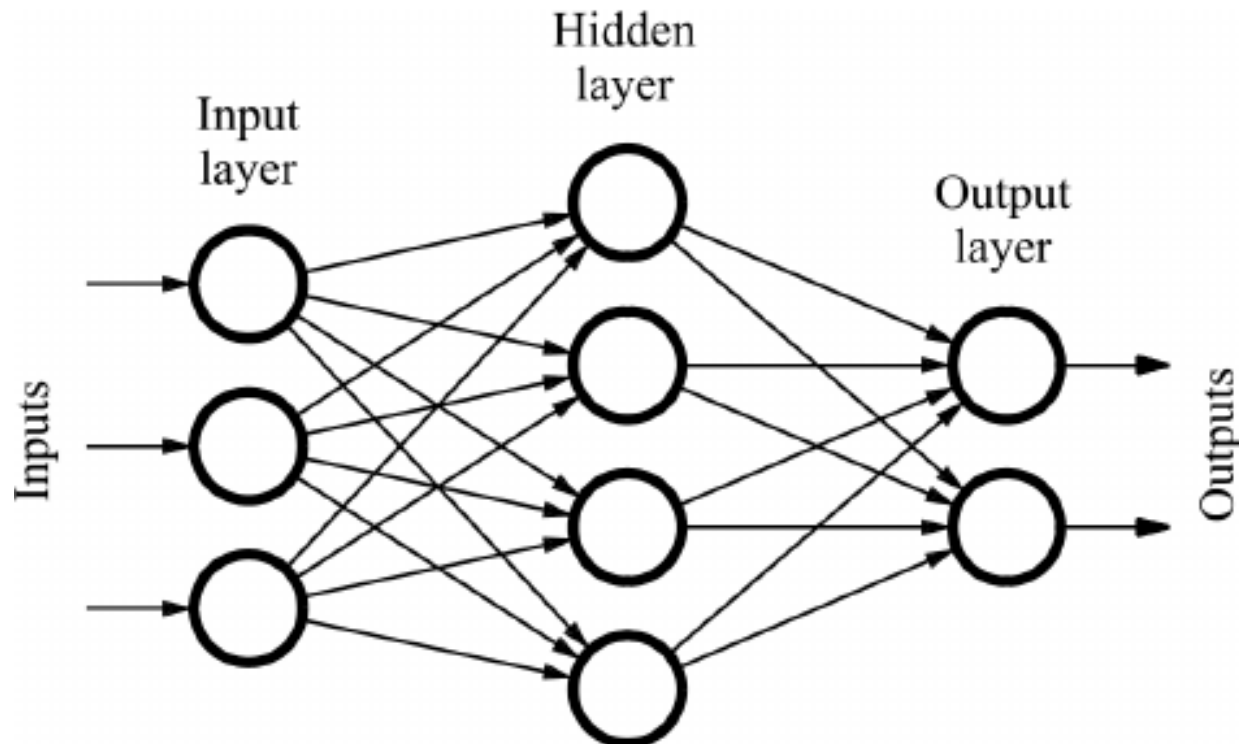
Simple Neural Model



NEURAL NETWORKS

BASIC CONCEPT

Multiple-Layer Neural Model



NEURAL NETWORKS

BASIC CONCEPT: NETWORK TOPOLOGY

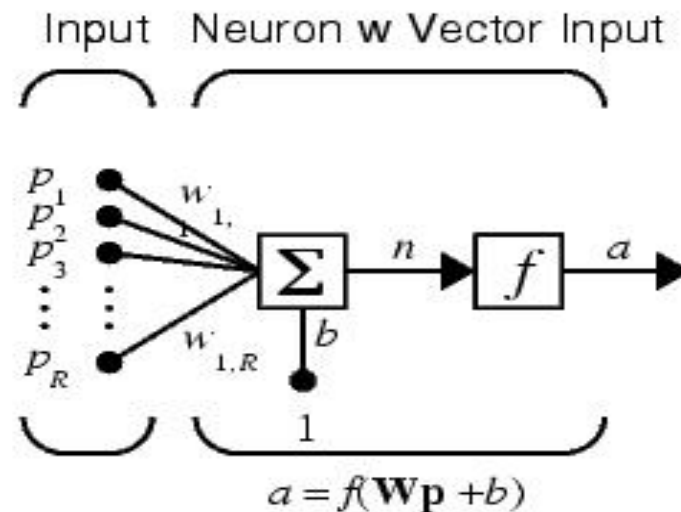
Network topology:

- Specify **number of units in the input layer**,
 - One input unit for each attribute
 - Normalize the input values for each attribute to [0.0—1.0]
- **number of hidden layers**,
- **number of units in each hidden layer**, and
- **number of units in the output layer**
 - if for classification and more than two classes, one output unit per class
- Once a network has been trained and its accuracy is unacceptable, repeat the training process with a different network topology or a different set of initial weights

NEURAL NETWORKS

BASIC CONCEPT: TRANSFER FUNCTION

Referring to the previous **Simple Neural Model**

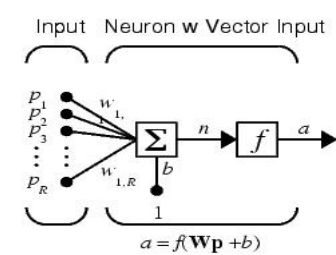


The sum output n , often referred to as the net input, goes into a **transfer function** f , also called **activation function**.

$$a = f(W * P + b) .$$

NEURAL NETWORKS

BASIC CONCEPT: TRANSFER FUNCTION



for instance if we have two inputs p_1 and p_2 . where $p_1 = 2$ and $p_2 = 3$, and the **connections'** weights of p_1 and p_2 are $w_1 = 1.5$ and $w_2 = 1$ respectively and $b = -1.5$, then

$$a = f(2 * (1.5) + 3 * (1) - 1.5) = f(4.5)$$

The **actual output depends** on the particular **transfer function** that is chosen.

- It is to be noted that many structures don't use bias.
 - In case bias b is used, its value with w keep changing based on the learning strategy used.

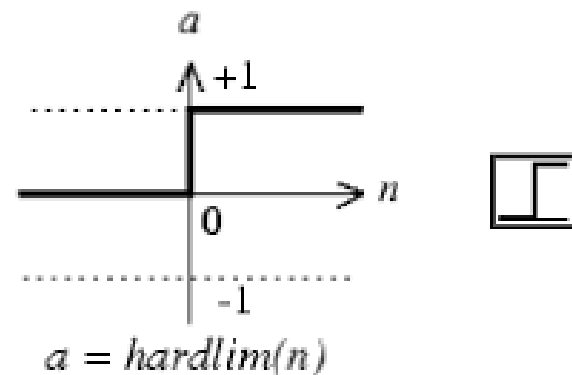
NEURAL NETWORKS

BASIC CONCEPT: TRANSFER FUNCTION

There are three main activation functions used commonly in neural networks:

- 1. Hard limit transfer function:** If the net input value n is above a certain threshold, the neuron becomes active (activation value of 1); otherwise it stays inactive (activation value of 0)

$$\varphi^{\text{hlim}}(v) = \begin{cases} 1 & \text{for } v \geq 0 \\ 0 & \text{for } v < 0 \end{cases}$$



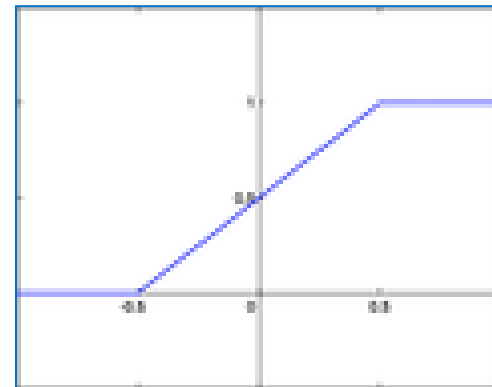
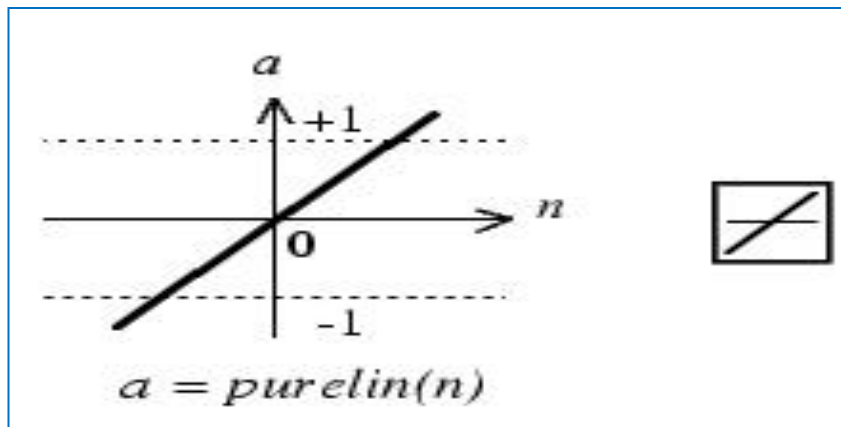
Hard-Limit Transfer Function

NEURAL NETWORKS

BASIC CONCEPT: TRANSFER FUNCTION

Transfer functions:

2. **Linear transfer/threshold function:** The activation increases linearly with the increase of the network input signal n , but after a certain threshold, the output becomes saturated (to a value of 1, say).



NEURAL NETWORKS

BASIC CONCEPT:

TRANSFER FUNCTION

Transfer functions:

3. **The sigmoid function.** This is any S-shaped nonlinear transformation function that is characterized by the following :

- a. Bounded, that is, its values are restricted between two boundaries
 - for example: $[0,1]$ or $[-1,1]$.
- b. Monotonically increased, that is, the value of the function never decreases when n increases.
- c. Continuous and smooth, therefore, differentiable everywhere in its domains.

NEURAL NETWORKS

BASIC CONCEPT:

TRANSFER FUNCTION

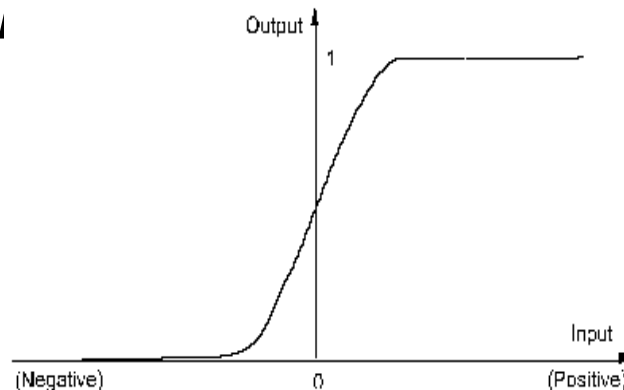
Transfer functions:

3. **The sigmoid function.** This is any S-shaped nonlinear transformation function that is characterized by the following :

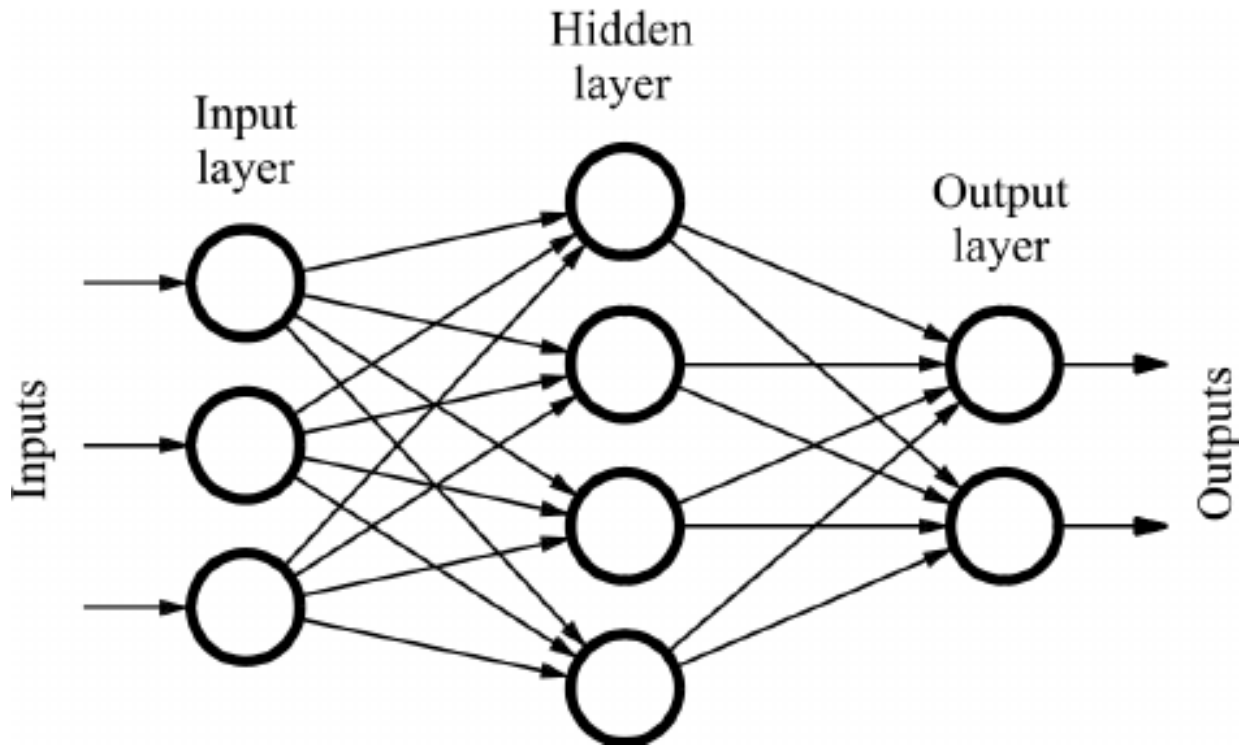
- Most of sigmoid functions are the logistic function

$$a = \frac{1}{1+e^{-n}} , w$$

$$-\infty \rightarrow [0,1]$$



A MULTI-LAYER FEED-FORWARD NN



HOW A MULTI-LAYER NN WORKS?

1. The inputs to the network correspond to the attributes measured for each training tuple
2. Inputs are fed simultaneously into the units making up the input layer
3. They are then weighted and fed simultaneously to a hidden layer
4. The number of hidden layers is arbitrary, although usually only one
5. The weighted outputs of the last hidden layer are input to units making up the output layer, which emits the network's prediction

HOW A MULTI-LAYER NN WORKS?

The network is feed-forward: None of the weights cycles back to an input unit or to an output unit of a previous layer

From a statistical point of view, networks perform nonlinear regression: Given enough hidden units and enough training samples, they can closely approximate any function

NEURAL NETWORKS AS A CLASSIFIER

Strength

- High tolerance to noisy data
- Ability to classify untrained patterns
- Well-suited for continuous-valued inputs and outputs
- Successful on an array of real-world data
 - e.g., hand-written letters
- Algorithms are inherently parallel
- Techniques have recently been developed for the extraction of rules from trained neural networks

Weakness

- Long training time
- Require a number of parameters typically best determined empirically, e.g., the network topology or “structure.”
- Poor interpretability: Difficult to interpret the symbolic meaning behind the learned weights and of “hidden units” in the network

MULTI-LAYER NEURAL NETWORKS

BACKPROPAGATION ALGORITHM

A Neural Network **learning algorithm**.

Iteratively process a set of training tuples and **compare the network's prediction** with the **actual known target** value.

For each training tuple, the **weights are modified** to **minimize the mean squared error** between the network's prediction and the actual target value.

Modifications are made in the “backwards” direction: from the output layer, through each hidden layer down to the first hidden layer, hence “backpropagation”

MULTI-LAYER NEURAL NETWORKS

BACKPROPAGATION ALGORITHM

Backpropagation Algorithm consists of two passes:

1. Forward pass

1. Apply an input vector X and its corresponding output vector Y (the desired output)
2. Propagate forward the input signals through all the neurons in all the layers and calculate the output signals.
3. Calculate the error for every output neuron

2. Backward pass

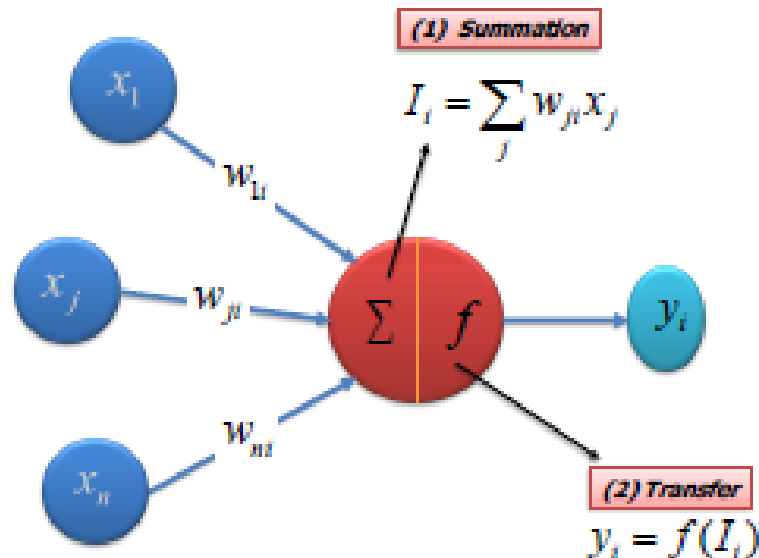
1. Adjust the weights between the intermediate neurons and output neurons j according to the calculated error.
2. Calculate the error for neurons in the intermediate layer
3. Propagate the error back to the neurons of lower level
4. Update each network weights

MULTI-LAYER NEURAL NETWORKS

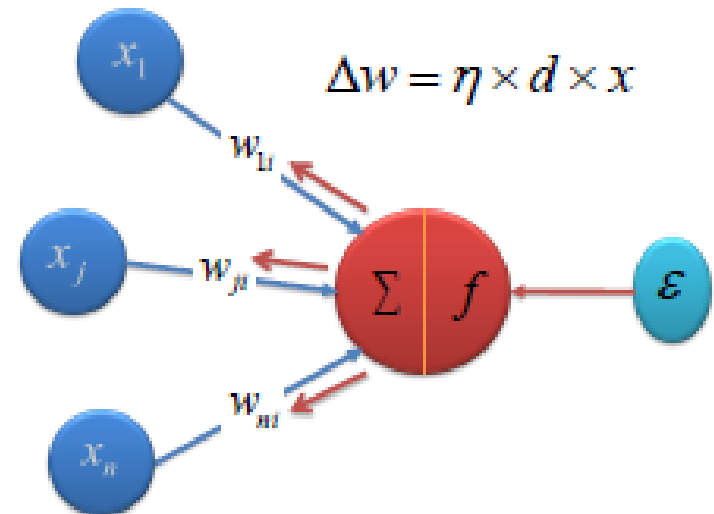
BACKPROPAGATION ALGORITHM

Backpropagation Algorithm consists of two passes:

Feedforward Input Data



Backward Error Propagation



NN – PYTHON'S LIBRARIES

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
iris = load_iris()

# extract only the lengths and widths of the petals:
X = iris.data[:, (2,3)]

# convert target to Setosa and Not Setosa (Versicolour and Virginica)
y = (iris.target==0).astype(np.int8)
# print(y)

from sklearn.neural_network import MLPClassifier
model = MLPClassifier(solver='lbfgs',
                      alpha=1e-5,
                      hidden_layer_sizes=(5, 2),
                      random_state=1)

model.fit(X, y)
result = model.predict([[0, 0], [1.8, 4],
                        [1, 0], [0, 1],
                        [1, 1], [2., 2.],
                        [1.3, 1.3], [2, 4.8]])

print(result)
```

MODEL EVALUATION

DO YOU REMEMBER THESE BASIC CONCEPTS?

Accuracy:

- refers to the ability of a given classifier to correctly predict the class label of new or previously unseen data

Speed:

- refers to the computational costs involved in generating and using the given classifier.

Robustness:

- refers to the ability of the classifier to make correct predictions given noisy data or data with missing values.

Scalability:

- refers to the ability to construct the classifier efficiently given large amounts of data.

Interpretability:

- refers to the level of understanding and insight that is provided by the classifier .
- Interpretability is subjective and therefore more difficult to assess.

CLASSIFICATION MODEL EVALUATION

Evaluating a classifier is often **significantly tricky**.

Accuracy is **the main evaluation metric** but **it is not the unique one**.

- use **test set** of labeled tuples instead of **training set** when assessing accuracy

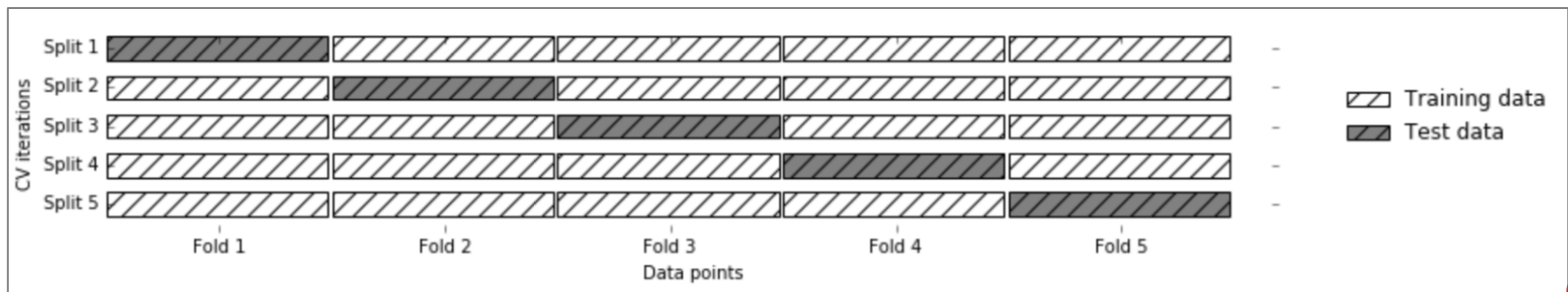
Methods for estimating a classifier's accuracy:

- **Holdout Method**, random subsampling
 - Training set and Test set
- **Cross-validation Method**

CLASSIFICATION MODEL EVALUATION

A good way to evaluate a model is to use **cross-validation**.

Cross-validation is a statistical method of **evaluating generalization performance** that is more stable and thorough than using a **split** into **a training** and **a test** sets.



- where k is a user-specified number of folds, usually 5 or 10.

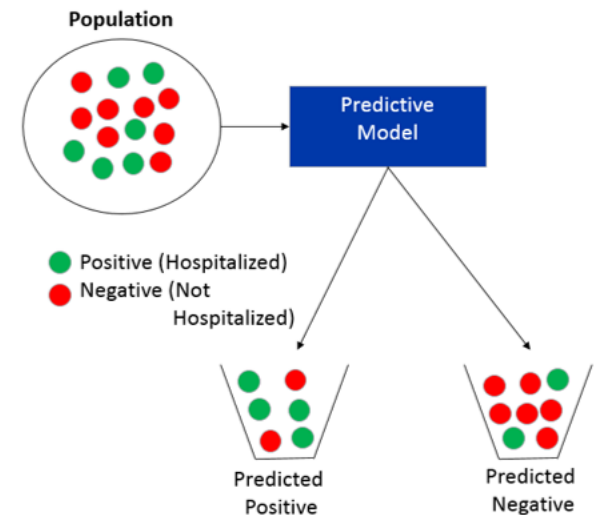
CLASSIFICATION MODEL EVALUATION

Confusion Matrix is another way to evaluate the performance of a classifier is to look at the *confusion matrix*.

- The general idea is to count the number of times that instances of **Class i** are classified as **Class j** .

		Predicted Class	
		Class 1	Class 2
Actual Class	Class 1	True Positives (TP)	False Negatives (FN)
	Class 2	False Positives (FP)	True Negatives (TN)

- May have extra rows/columns to provide totals



CLASSIFIER EVALUATION METRICS

Classifier Accuracy, or recognition rate

- is the percentage of test set tuples that are correctly classified

$$\text{Accuracy} = \frac{TP+TN}{All}$$

$(P \rightarrow)$ $(A \downarrow)$	C_1	C_2	
C_1	TP	FN	P
C_2	FP	TN	N
	P'	N'	All

Error rate: $1 - \text{Accuracy}$, or

$$\text{Error rate} = \frac{FP+FN}{All}$$

CLASSIFIER EVALUATION METRICS

Class Imbalance Problem:

- One class may be rare
 - e.g. fraud
- Significant majority of the negative class and minority of the positive class
- **Sensitivity**: True Positive recognition rate
 - Sensitivity = $\frac{TP}{P}$
- **Specificity**: True Negative recognition rate
 - Specificity = $\frac{TN}{N}$

$(P \rightarrow)$ $(A \downarrow)$	C_1	C_2	
C_1	TP	FN	P
C_2	FP	TN	N
	P'	N'	All

CLASSIFIER EVALUATION METRICS

Precision: **exactness** – the ratio of tuples that the classifier labeled as positive are actually positive, perfect score is 1.0.

- It is know as

$(P \rightarrow)$ $(A \downarrow)$	C_1	C_2	
C_1	TP	FN	P
C_2	FP	TN	N
	P'	N'	All

$$\text{Precision} = \frac{TP}{TP + FP}$$

CLASSIFIER EVALUATION METRICS

Recall: completeness – the ratio of positive tuples that the are correctly classified as positive, perfect score is 1.0

- It is known as *sensitivity*

$$\text{Recall} = \frac{TP}{TP+FN}$$

$(P \rightarrow)$ $(A \downarrow)$	C_1	C_2	
C_1	TP	FN	P
C_2	FP	TN	N
	P'	N'	All

SUPERVISED LEARNING CLASSIFICATION MODEL - EVALUATION

F measure (F1 or F1-score): harmonic mean metric that to precision and recall into a single metric.

- *F1 Score* inverses relationship between The Precision and the Recall of a classifier
- *F1 Score almost used to compare two classifiers.*

$$\bullet F1 = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

$$\bullet F1 = \frac{TP}{TP + \frac{FN + FP}{2}}$$

$(P \rightarrow)$ $(A \downarrow)$	C_1	C_2	
C_1	TP	FN	P
C_2	FP	TN	N
	P'	N'	All

CLASSIFIER EVALUATION METRICS

Assume that we get the following confusion matrix for a certain classifier:

$(P \rightarrow)$ $(A \downarrow)$	cancer = yes	cancer = no	Total	Recognition(%)
cancer = yes	90	210	300	30.00 (<i>sensitivity</i>)
cancer = no	140	9560	9700	98.56 (<i>specificity</i>)
Total	230	9770	10000	96.50 (<i>accuracy</i>)

- **Accuracy** = $(90+9560)/1000 = 96.5\%$
- Precision and Recall for the class **cancer=yes**
 - Precision = $90/230 = 39.13\%$
 - Recall = $90/300 = 30.00\%$

CLASSIFIER EVALUATION METRICS

Assume that we get the following confusion matrix for a certain classifier:

$(P \rightarrow)$ $(A \downarrow)$	cancer = no	cancer = yes	Total	Recognition(%)
cancer = no	9560	140	9700	98.56 (<i>sensitivity</i>)
cancer = yes	210	90	300	30.00 (<i>specificity</i>)
Total	9770	230	10000	96.50 (<i>accuracy</i>)

- **Accuracy** = $(90+9560)/1000 = 96.5\%$
- Precision and Recall for the class **cancer=no**
 - Precision = $9560/9770 = 97.85\%$
 - Recall = $9560/9700 = 98.56\%$

CLASSIFIER EVALUATION– PYTHON'S LIBRARIES

```
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
iris = load_iris()
# Import train_test_split function
from sklearn.model_selection import train_test_split

# Split dataset into 70% training set and 30% test set
X_train, X_test, y_train, y_test =
    train_test_split(iris.data, iris.target, test_size=0.3)

# Naive Bayes
from sklearn.naive_bayes import GaussianNB as gnb
model = gnb()
model.fit(X_train, y_train)

y_pred=model.predict(X_test)

#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics
# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", metrics.confusion_matrix(y_test, y_pred))
print("Classification Report:\n", metrics.classification_report((y_test,
y_pred)))
```

ASSIGNMENT III

Compare the behavior of three distinct classifiers on your own dataset.

- Classifier behavior can be determined by evaluation metrics such as: Classifier's Accuracy and Precision, Recall and F-measure for each Class in your dataset.

Notes

- You can use any three classifier
- Submit the Python code all the used classifiers
- Report the behavior of the classifiers in Word's document that describes our experiment.
- Deadline: Friday 25 MAR, 2022 23:55